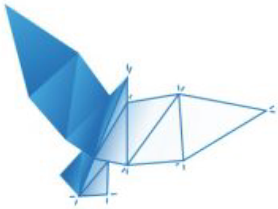


NVISO Labs

Cyber security research, straight from the lab! 🐀

Menu

Intercepting Flutter traffic on Android (ARMv8)

Jeroen Beckers Mobile Security, Android May 20, 2020 7 Minutes

In a previous blogpost, I explained my steps for reversing the flutter.so binary to identify the correct offset/pattern to bypass certificate validation. As a very quick summary: **Flutter doesn't use the system's proxy settings, and it doesn't use the system's certificate store**, so normal approaches don't work. My previous guide only explained how to intercept Flutter on ARMv7 Android devices, but the steps don't fully transfer to ARMv8 so this blogpost quickly explains the steps for ARMv8

> ⚠️ **Update August 2022** ⚠️
>
> An update to this blog post was written and can be found here. It covers both iOS and Android and a convenient script / Frida codeshare to use.

This blogpost is written as a guide / thought process, so you can find a **TL;DR at the bottom**.

Testing apps

First, we'll need a testing app. I've slightly updated the previous one to have two buttons: one for HTTP and one for HTTPS calls. This way, I can validate whether the proxy works, and then whether the Frida script works.

The app can be downloaded from our GitHub.

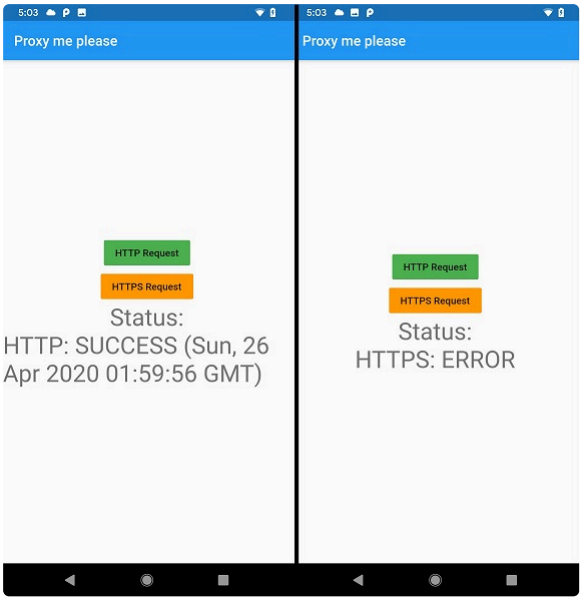
There are two functions in the app that call an HTTP and HTTPS endpoint:

```
void callHTTP(){
  client = HttpClient();
  _status = "Calling...";
  client
      .getUrl(Uri.parse('http://neverssl.com'))
      .then((request) => request.close())
      .then((response) => setState((){_status = "HTTP:
      .catchError((e) =>
          setState(() {
            _status = "HTTP: ERROR";
            print(e.toString());
          })
      );
}
void callHTTPS(){
  client = HttpClient();
  _status = "Calling...";
  client
      .getUrl(Uri.parse('https://www.nviso.eu')) // pro
      .then((request) => request.close()) // sends the
      .then((response) => setState((){
                                          _status = "HTTP
                                      }))
      .catchError((e) =>
                      setState(() {
                        _status = "HTTPS: ERROR";
                        print(e.toString());
                      })
                  );
}
```

Proxying the application

Flutter applications still don't automatically use the system's proxy, unless the developer adds this functionality by creating custom Android & iOS plugins that provide this information. Obviously, many developers won't do this, so we still need

to intercept the traffic using ProxyDroid's root-based method rather than configuring the WIFI's proxy through the Android OS. After configuring ProxyDroid with the correct settings, Burp can see the requests from the app.



The HTTP requests work without any special requirement, while the HTTPS call prints an error to logcat:

```
04-26 16:59:02.758 11773 11802 E flutter : [ERROR:flutter/li
04-26 16:59:02.758 11773 11802 E flutter :       NO_START_LIN
04-26 16:59:02.758 11773 11802 E flutter :       PEM routines
04-26 16:59:02.758 11773 11802 E flutter :       NO_START_LIN
04-26 16:59:02.758 11773 11802 E flutter :       PEM routines
04-26 16:59:02.758 11773 11802 E flutter :       CERTIFICATE_
```

Disabling SSL verification

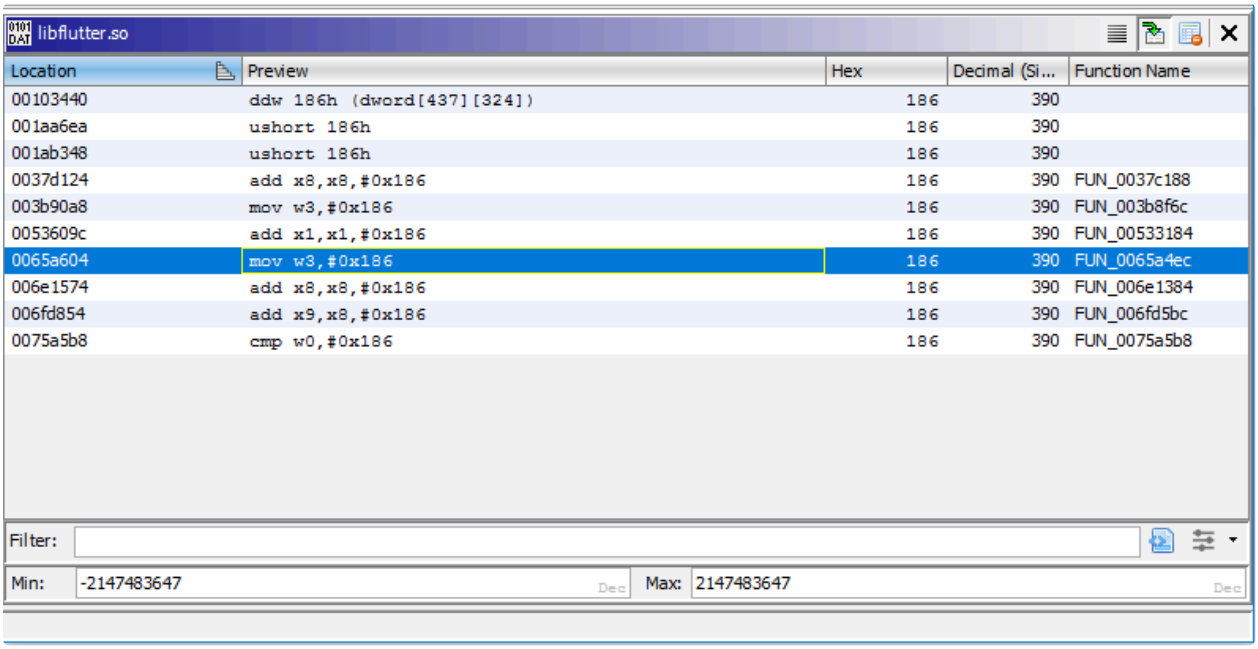
I initially thought the x64 version would be identical to the x86 version. It's the same source code, so why would the steps be any different… Unfortunately, when searching for 'x509.cc' in flutter.so, I found the same number of hits, but none of them were the correct function:

```
                    s_../../third_party/boringssl/src/_001c885a     XREF[4]:     FUN_005f4e1c:005f65f4(*),
                                                                                 FUN_0065a224:0065a3a0(*),
                                                                                 FUN_0065a224:0065a3b4(*),
                                                                                 FUN_0065a224:0065a400(*)
001c885a 2e 2e 2f        ds         "../../third_party/boringssl/src/ssl/ssl_x509....
         2e 2e 2f
         74 68 69 ...
```

The previous approach seems inefective

It's pretty obvious that the ssl_x509.cc class has been compiled somewhere in the 0x650000 region, but that's still a lot of functions to try to find the correct one. If

searching for the filename doesn't work, maybe searching for the line number would work. If we take a look at the ssl_crypto_x509_session_verify_cert_chain function again, we can see that the OPENSSL_PUT_ERROR macro is called at line 390. Searching for the number 390 (or 0x186) gives us some results (Search > For Scalars…):



Searching for magic numbers

A few of the results are around the 0x650000 region. The highlighted function (FUN_0065a4ec) looks like a good candidate, as the constant is loaded in w3 (the lower 32bit part of the x3 register), which is one of the argument registers on ARMv8. The function FUN_0065a4ec also has the correct signature, and it generally looks the same as the ARMv7 version:

Decompiled method in x86 vs x64

My normal approach would be to copy the first bytes of FUN_0065a4ec and search for them in-memory while the application is running, as I did in the previous blogpost, so I don't need to find the offset each time. Unfortunately, Frida's Memory.scan seems to crash on my test app, so for now we'll have to use the offset. (*Edit: An alternative approach was posted in the comments of this post, using Process.enumerateRangesSync*)

Ghidra uses 0x100000 as the base address of the module, so we have to subtract that from the Ghidra offset, resulting in an offset of **0x55a4ec**.

Opening Ghidra every time works, but it's not that convenient. We can also use binwalk to find the correct offset based on those first bytes of the function:

```
# The first bytes of the FUN_0065a4ec function
ff 03 05 d1 fc 6b 0f a9 f9 63 10 a9 f7 5b 11 a9 f5 53 12
# Find it using binwalk
binwalk -R "\xff\x03\x05\xd1\xfc\x6b\x0f\xa9\xf9\x63\x10
DECIMAL       HEXADECIMAL     DESCRIPTION
--------------------------------------------------------
5612780       0x55A4EC        Raw signature (\xff\x03\x0
```

Let's throw this in a Frida script and test it!

```
function hook_ssl_verify_result(address)
{
  Interceptor.attach(address, {
```

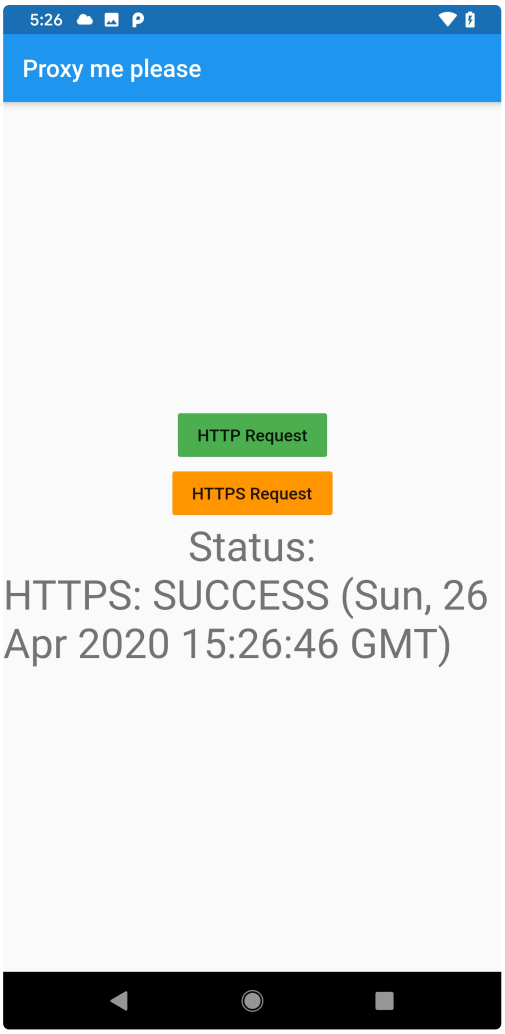
```
    onEnter: function(args) {
      console.log("Disabling SSL validation")
    },
    onLeave: function(retval)
    {
      console.log("Retval: " + retval)
      retval.replace(0x1);

    }
  });
}
function disablePinning(){
    // Change the offset on the line below with the bin
    // If you are on 32 bit, add 1 to the offset to ind
    // Otherwise, you will get  'Error: unable to inter
    var address = Module.findBaseAddress('libflutter.so
    hook_ssl_verify_result(address);
}
setTimeout(disablePinning, 1000)
```

Running this file using Frida gives the expected outcome:

```
(secenv) ➜  flutter frida -Uf be.nviso.flutter_app -l h

     ____
    / _  |   Frida 12.8.20 - A world-class dynamic inst
   | (_| |
    > _  |   Commands:
   /_/ | |       help      -> Displays the help system
   . . . .       object?   -> Display information about
   . . . .       exit/quit -> Exit
   . . . .
   . . . .   More info at https://www.frida.re/docs/hom
Spawned `be.nviso.flutter_app`. Resuming main thread!
[SM-G950F::be.nviso.flutter_app]-> disablePinning()
[SM-G950F::be.nviso.flutter_app]-> Disabling SSL valida
Retval: 0x0
[SM-G950F::be.nviso.flutter_app]->
```



SSL Verification successfully disabled

Tangent: Why can this app perform cleartext HTTP calls?

My flutter app is making HTTP connections. This is forbidden by default since Android P, and you have to add a Network Security Policy that explicitly allows cleartext request if you still want to do so on Android 9+. My test app does not have a Network Security Policy, so what's going on?

The reason for this is the same reason why these blog posts are necessary: Flutter doesn't use default Android libraries. Because Flutter creates low level sockets and implements the HTTP stack on top of that, the requests never pass by the Android security controls that should prevent cleartext traffic from being used. This is an important thing to keep in mind when auditing the security of Flutter apps, as you might miss things if you're not careful.

TL;DR (ARMv7 and ARMv8)

1. Redirect with ProxyDroid on rooted device since Flutter apps are still proxy-unaware

2. Find the offset using binwalk

3. Use the Frida script to hook the method at that offset

Since the last blogpost, the signature for 32bit also changed, so I've included both signatures.

```
# Method signatures for ARMv7 (32bit)
2d e9 f0 4f a3 b0 81 46 50 20 10 70
2d e9 f0 4f a3 b0 82 46 50 20 10 70
# Get the offset
binwalk -R "\x2d\xe9\xf0\x4f\xa3\xb0\x81\x46\x50\x20\x1
DECIMAL       HEXADECIMAL     DESCRIPTION
--------------------------------------------------------
3831160       0x3A7578        Raw signature (\x2d\xe9\x
# Method signature for ARMv8 (64bit)
ff 03 05 d1 fc 6b 0f a9 f9 63 10 a9 f7 5b 11 a9 f5 53 1
# Get the offset
binwalk -R "\xff\x03\x05\xd1\xfc\x6b\x0f\xa9\xf9\x63\x1
DECIMAL       HEXADECIMAL     DESCRIPTION
--------------------------------------------------------
5612780       0x55A4EC        Raw signature (\xff\x03\x
```

Frida script to use the offset:

```
function hook_ssl_verify_result(address)
{
  Interceptor.attach(address, {
    onEnter: function(args) {
      console.log("Disabling SSL validation")
    },
    onLeave: function(retval)
    {
      console.log("Retval: " + retval)
      retval.replace(0x1);

    }
  });
}
function disablePinning(){
    // Change the offset on the line below with the bin
    // If you are on 32 bit, add 1 to the offset to ind
    // Otherwise, you will get  'Error: unable to inter
    var address = Module.findBaseAddress('libflutter.so
    hook_ssl_verify_result(address);
}
setTimeout(disablePinning, 1000)
```

And launch it using Frida:

```
frida -Uf hook.js -f be.nviso.flutter_app --no-pause
```

If it still doesn't work, you'll have to figure out the correct method to hook yourself. You can try following the steps for ARMv7 as described on this blog.

About the author

Jeroen Beckers is a mobile security expert working in the NVISO Cyber Resilience team and co-author of the OWASP Mobile Security Testing Guide (MSTG). He also loves to program, both on high and low level stuff, and deep diving into the Android internals doesn't scare him. You can find Jeroen on LinkedIn.

Tagged: Mobile, android, flutter

Published by Jeroen Beckers

View all posts by Jeroen Beckers

‹ Three tips for a better IT Acceptable Use Policy

A checklist to populate your Acceptable Use Policy ›

21 thoughts on "Intercepting Flutter traffic on Android (ARMv8)"

> Pingback: Intercepting traffic from Android Flutter applications – NVISO Labs

AaYohan

June 6, 2020 at 11:16 pm

Hi Jeroen,

First of all, thanks you for sharing.

I have workaround for sigscan on arm64, rather than using module i’m using memory range as base address and size.

frida –version

12.9.4

```
Process.enumerateRangesSync('r-x').filter(function (m) {
if (m.file)
return m.file.path.indexOf('libflutter.so') > -1;
return false;
}).forEach(function (r) {
Memory.scanSync(r.base, r.size, pattern).forEach(function (match) {
hook_ssl_verify_result(match.address);
});
});
```

Loading...

↳ Reply

Jeroen Beckers

June 8, 2020 at 8:47 am

Thanks for the alternative suggestion! That’s a good workaround until the bug is fixed at Frida’s side.

Loading...

↳ Reply

Pingback: Intercepting Flutter traffic on iOS – NVISO Labs

Andrew Romanov

August 31, 2020 at 4:46 pm

The method doesn't work, even with your test app

Loading...

↪ Reply

Jeroen Beckers

August 31, 2020 at 7:40 pm

I've just retested both 32bit and 64bit ARM, so if the test app doesn't work, it must be something with your device.

Loading...

↪ Reply

Andrew Romanov

September 1, 2020 at 9:38 am

What device do you have?

Loading...

BRUHItsABunny

January 5, 2021 at 9:39 pm

Disclaimer: I'm a noob when it comes to reversing native binaries

I was targeting a 64 bit .so and ended up searching for the string: "ssl_server" instead to find the right function

Inspiration: https://github.com/google/boringssl/blob/78f15a6aa9f11ab7cff736f920c4858cc38264fb/ssl/ssl_x509.cc#L386

The string was found to be defined only once so it was easy to xref it, which gave me 3 references (twice in the target function and once in the rodata where it was defined).

Also ended up using AaYohan's way of hooking since Module.findBaseAddress('libflutter.so') kept returning null for me.

Not sure why it worked, just happy it worked.

Loading...

↳ Reply

Pingback: Intercept Android SSL Pinning on Flutter Based Android Application – Qlkwej

Alex

June 6, 2021 at 3:27 pm

Thankyou for this post!

I applied a similar procedure for x86_64 architecture which has different method signatures and it worked perfectly

Loading...

↳ Reply

enovella

June 30, 2021 at 10:16 am

Searching the offset with Radare2 is a piece of cake:

$ r2 ~/Downloads/flutterCertPinning.apk.unpack/lib/arm64-v8a/libflutter.so
Warning: run r2 with -e bin.cache=true to fix relocations in disassembly
— Of course r2 runs FreeBSD
[0x00270000]> /x ff 03 05 d1 fc 6b 0f a9 f9 63 10 a9 f7 5b 11 a9 f5 53 12 a9 f3 7b 13 a9 08 0a 80 52
Searching 28 bytes in [0x816d00-0x828710]
hits: 0
Searching 28 bytes in [0x6f0000-0x816d00]
hits: 0
Searching 28 bytes in [0x270000-0x6e1ce0]
hits: 1
Searching 28 bytes in [0x0-0x26cc9c]
hits: 0
0x0055a4ec hit0_0
ff0305d1fc6b0fa9f96310a9f75b11a9f55312a9f37b13a9080a8052
[0x00270000]>

Loading...

↳ Reply

Pingback: Null address in Sslpinning bypass of flutter app by using frida – Ask Android Questions

Edward Guan

October 6, 2021 at 4:56 pm

Hello Jeroen,

I have a question about `binwalk` and `Ghidra`. I got two different address by these tools:

https://imgur.com/a/OUsceBu

I have changed the base address to 0x0 for Ghidra. Do you know why the binwalk command got the wrong address? Thanks very much.

Loading...

↳ Reply

Pingback: Proxying Android app traffic – Common issues / checklist (2023) – NVISO Labs

luke

April 13, 2023 at 12:39 pm

Thanks for the post.

I didn't understand this part:
"we can see that the OPENSSL_PUT_ERROR macro is called at line 390. Searching for the number 390 (or 0x186) gives us some results (Search > For Scalars…):"
Why would we deduce that the line number on a c++ program would be present in the binary of the application?

Loading...

↳ Reply

luke

April 13, 2023 at 12:42 pm

Nevermind, I understand now that the OPENSSL_PUT_ERROR macro will need the line number where the error ocurred.

Loading...

↳ Reply

Carnival

May 10, 2023 at 11:14 am

Using the APK you provided on Github, but using bingwalk cannot find a corresponding bytecode or any offset

Loading...

Reply

Jeroen Beckers

May 10, 2023 at 11:19 am

Please try the latest version of the script on https://github.com/NVISOsecurity/disable-flutter-tls-verification . If it still doesn't work, please open a github issues.

Loading...

Reply

Carnival

May 10, 2023 at 11:41 am

sorry read others blog done

Loading...

Reply

Dastan

January 28, 2024 at 10:50 am

TypeError: cannot read property ‘add’ of null
at disablePinning
Getting this error when firing this script.

On further investigation of modules being loaded using the below code.

Process.enumerateModules({
onMatch: function(module){
console.log(‘Module name: ‘ + module.name + ” – Base Address: ” +
module.base.toString());
},
onComplete: function(){}
});

It was observed that libflutter.so is not there.

Loading...

↳ Reply

Jeroen Beckers

January 29, 2024 at 9:51 am

Hi Dastan! Please open a ticket on the github https://github.com/NVISOsecurity/disable-flutter-tls-verification and include the APK that isn’t working.

Loading...

↳ Reply

Leave a Reply

NVISO Homepage

Jobs

Info and support

info@nviso.eu

Got hacked?

csirt@nviso.eu